OAVS TGT COMPUTER TEACHER EXAM

MCQ Practice Book

Header Files in C

Complete Question Bank with Explanations & Answer Key

125 Multiple Choice Questions

Standard Headers · User-Defined Headers · Library Functions · Conceptual

Prepared: May 2026

125 Questions	4 Sections	Answer Key	Explanations

Table of Contents

Section 1	Standard Header Files	*Q1 – Q25*
Section 2	User-Defined Header Files	*Q26 – Q50*
Section 3	Uses of Header Files & Library Functions	*Q51 – Q100*
Section 4	Advanced & Conceptual Questions	*Q101 – Q125*
Appendix A	Quick Reference — Header Files Summary Table	
Appendix B	Complete Answer Key	
Appendix C	Detailed Explanations	

Exam Instructions & Guidelines

Exam Pattern: This book contains 125 MCQs covering all aspects of Header Files in C, aligned with the OAVS TGT Computer Teacher syllabus.

Question Types: Direct knowledge (syntax, declarations), Matching (header ↔ function), Application (what does X return), and Conceptual/analytical questions.

Marking Scheme: Each question carries 1 mark. No negative marking is specified for OAVS TGT. Confirm with official notification.

Answer Key: A complete answer key is provided in Appendix B. Detailed explanations for every question are in Appendix C.

Study Advice: Attempt all questions first, then check answers. Review explanations for all incorrect answers. Focus on Section 3 (Library Functions) as these are frequently tested.

Time Suggestion: Allow 60–75 seconds per question for practice (equivalent to 125–150 minute full mock). For quick revision, use Section headers to focus weak areas.

Section 1: Standard Header Files *Questions 1–25*

Q1. 1. Which header file is required to use the printf() and scanf() functions in C?

A) stdlib.h
B) stdio.h
C) string.h
D) math.h

■ **Correct Answer: B) stdio.h**
Explanation: stdio.h stands for Standard Input/Output. It declares printf(), scanf(), fprintf(), fscanf(), fopen(), fclose() and all I/O-related functions. It is the most commonly included header in C programs.

Q2. 2. What does the acronym stdio.h stand for?

A) String Data Operations
B) Standard Data Input/Output
C) Standard Input/Output
D) System Data Interface Operations

■ **Correct Answer: C) Standard Input/Output**
Explanation: stdio.h literally expands to 'Standard Input/Output header file'. It provides the standard set of I/O facilities for C programs.

Q3. 3. Which header file provides the sqrt() and pow() functions?

A) stdlib.h
B) stdio.h
C) math.h
D) conio.h

■ **Correct Answer: C) math.h**
Explanation: math.h declares mathematical functions: sqrt(), pow(), floor(), ceil(), sin(), cos(), tan(), fabs(), log(), exp() etc. You must also link the math library with -lm on Linux.

Q4. 4. The function strlen() is declared in which header file?

A) stdio.h
B) stdlib.h
C) ctype.h
D) string.h

■ **Correct Answer: D) string.h**
Explanation: string.h provides string-handling functions: strlen(), strcpy(), strcat(), strcmp(), strchr(), strstr(), memset(), memcpy() etc.

Q5. 5. Which header file must be included to use malloc() and free()?

A) alloc.h
B) memory.h
C) stdlib.h
D) stdio.h

■ **Correct Answer: C) stdlib.h**
Explanation: stdlib.h (Standard Library) provides memory management (malloc, calloc, realloc, free), process control (exit, abort), integer arithmetic (abs, div), and string conversion (atoi, atof, strtol).

Q6. 6. The isalpha() and isdigit() functions are declared in:

A) string.h
B) ctype.h
C) stdio.h
D) stdlib.h

■ **Correct Answer: B) ctype.h**
Explanation: ctype.h (Character Type) declares character classification functions: isalpha(), isdigit(), isalnum(), isspace(), isupper(), islower(), toupper(), tolower() etc.

Q7. 7. Which header file is used for time-related functions like time() and clock()?

A) sys.h
B) timer.h
C) date.h
D) time.h

■ **Correct Answer: D) time.h**
Explanation: time.h declares the time_t type, struct tm, and functions: time(), clock(), difftime(), mktime(), strftime(), localtime(), gmtime() for date/time manipulation.

Q8. 8. To use assert() macro in C, which header file is needed?

A) debug.h
B) error.h
C) assert.h
D) check.h

■ **Correct Answer: C) assert.h**
Explanation: assert.h provides the assert() macro used for debugging. When NDEBUG is defined, assert() is disabled, making it safe to leave assertions in production code.

Q9. 9. The errno variable and error codes are declared in:

A) error.h
B) errno.h
C) stdio.h
D) stdlib.h

■ **Correct Answer: B) errno.h**
Explanation: errno.h declares the errno variable and error code macros like EDOM, ERANGE, ENOMEM etc. Functions set errno to indicate error conditions.

Q10. 10. Which header file provides signal handling in C?

A) handle.h
B) interrupt.h
C) signal.h
D) event.h

■ **Correct Answer: C) signal.h**
Explanation: signal.h declares signal(), raise(), and signal constants like SIGINT, SIGTERM, SIGFPE, SIGSEGV for handling system signals.

Q11. 11. The va_list, va_start, and va_end macros used for variable argument lists are in:

A) args.h
B) vararg.h
C) stdarg.h
D) arglist.h

■ **Correct Answer: C) stdarg.h**
Explanation: stdarg.h provides the mechanism for functions with variable number of arguments (variadic functions). It defines va_list type and va_start(), va_arg(), va_end() macros.

Q12. 12. Which header file contains the NULL macro and size_t type?

A) null.h
B) types.h
C) stddef.h
D) define.h

■ **Correct Answer: C) stddef.h**
Explanation: stddef.h defines common macros and types: NULL (null pointer constant), size_t (unsigned type for sizes), ptrdiff_t (pointer difference type), and offsetof() macro.

Q13. 13. The function exit() for program termination is declared in:

A) stdio.h
B) stdlib.h
C) process.h
D) system.h

■ **Correct Answer: B) stdlib.h**
Explanation: stdlib.h declares exit(), atexit(), and abort(). EXIT_SUCCESS (0) and EXIT_FAILURE (1) constants are also defined there.

Q14. 14. Which header file provides setjmp() and longjmp() for non-local jumps?

A) jump.h
B) goto.h
C) setjmp.h
D) jmp.h

■ **Correct Answer: C) setjmp.h**
Explanation: setjmp.h provides setjmp() and longjmp() for implementing non-local jumps (exception-like handling). setjmp() saves the calling environment, and longjmp() restores it.

Q15. 15. The FILE data type and file operations are defined in which header?

A) file.h
B) fio.h
C) fileop.h
D) stdio.h

■ **Correct Answer: D) stdio.h**
Explanation: The FILE type, along with fopen(), fclose(), fread(), fwrite(), fseek(), ftell(), rewind() and macros like EOF, SEEK_SET, SEEK_CUR, SEEK_END are all in stdio.h.

Q16. 16. Which header provides integer limits like INT_MAX, INT_MIN, CHAR_MAX?

A) values.h
B) maxmin.h
C) limits.h
D) constants.h

■ **Correct Answer: C) limits.h**
Explanation: limits.h defines implementation-specific limits for integer types: CHAR_BIT, CHAR_MIN, CHAR_MAX, SHRT_MAX, INT_MAX, INT_MIN, LONG_MAX, ULONG_MAX etc.

Q17. 17. The float limits FLT_MAX, DBL_MAX, and FLT_EPSILON are in:

A) float.h
B) limits.h
C) math.h
D) double.h

■ **Correct Answer: A) float.h**
Explanation: float.h defines the characteristics of floating-point types: FLT_MAX, FLT_MIN, FLT_EPSILON (for float), DBL_MAX, DBL_MIN, DBL_EPSILON (for double), LDBL_MAX etc.

Q18. 18. Which header file is used for locale-specific behavior like setlocale()?

A) region.h
B) locale.h
C) country.h
D) location.h

■ **Correct Answer: B) locale.h**
Explanation: locale.h provides setlocale() and localeconv() for locale-specific formatting of numbers, dates, and currencies. LC_ALL, LC_NUMERIC, LC_TIME are defined here.

Q19. 19. To use the wchar_t wide-character type, which header is needed?

A) char.h
B) unicode.h
C) wchar.h
D) wide.h

■ **Correct Answer: C) wchar.h**
Explanation: wchar.h (added in C95/C99) provides wide-character types and functions: wchar_t, wint_t, fwprintf(), fwscanf(), wcslen(), wcscpy(), wcscat() etc.

Q20. 20. The stdbool.h header provides which feature?

A) Stack-based boolean operations
B) Boolean type bool and constants true/false
C) Bitwise boolean operations
D) Boolean string functions

■ **Correct Answer: B) Boolean type bool and constants true/false**
Explanation: stdbool.h (C99) defines the _Bool type (aliased as bool) and the constants true (1) and false (0), enabling C programs to use boolean variables naturally.

Q21. 21. Which header file contains the int8_t, int16_t, int32_t, int64_t fixed-width integer types?

A) types.h
B) sys/types.h
C) stdint.h
D) inttypes.h

■ **Correct Answer: C) stdint.h**
Explanation: stdint.h (C99) declares fixed-width integer types: int8_t, uint8_t, int16_t, uint16_t, int32_t, uint32_t, int64_t, uint64_t — ensuring consistent sizes across platforms.

Q22. 22. The complex.h header (C99) provides support for:

A) Complex control structures
B) Complex data structures
C) Complex number arithmetic
D) Complex string operations

■ **Correct Answer: C) Complex number arithmetic**
Explanation: complex.h provides support for complex number arithmetic: complex type, creal(), cimag(), cabs(), carg(), cadd(), csub(), cmul(), cdiv() etc. It requires C99 or later.

Q23. 23. Which header file supports printing format macros like PRId32, PRIu64?

A) format.h
B) inttypes.h
C) stdint.h
D) printf.h

■ **Correct Answer: B) inttypes.h**
Explanation: inttypes.h (C99) extends stdint.h by adding format macros for printf/scanf: PRId8, PRId16, PRId32, PRId64, PRIu32 etc., ensuring portable formatted I/O of fixed-width integers.

Q24. 24. The conio.h header file provides functions like getch() and clrscr(). This header is:

A) Part of ANSI C standard
B) Part of C99 standard
C) A non-standard, compiler-specific header
D) Part of POSIX standard

■ **Correct Answer: C) A non-standard, compiler-specific header**
Explanation: conio.h (Console I/O) is NOT part of the ANSI/ISO C standard. It is Borland/Turbo C specific and not available in GCC/MSVC by default. getch(), clrscr(), gotoxy() are its common functions.

Q25. 25. Which of the following is the correct way to include a standard header file?

A) #include {stdio.h}
B) #include [stdio.h]
C) #include
D) #include (stdio.h)

■ **Correct Answer: C) #include**
Explanation: Standard header files are included using angle brackets < >: #include . This tells the preprocessor to search in system/standard include directories. User-defined headers use double quotes.

Section 2: User-Defined Header Files *Questions 26–50*

Q26. 26. How is a user-defined header file included in a C program?

A) #include
B) #include "myfile.h"
C) #import "myfile.h"
D) #use myfile.h

■ **Correct Answer: B) #include "myfile.h"**
Explanation: User-defined header files are included using double quotes: #include "myfile.h". The preprocessor first searches the current directory, then the system include directories.

Q27. 27. What is the recommended file extension for user-defined header files in C?

A) .c
B) .cpp
C) .h
D) .hdr

■ **Correct Answer: C) .h**
Explanation: The conventional extension for C header files is .h (e.g., myutils.h). This convention is universally recognized by C compilers and tools, though any extension technically works.

Q28. 28. Which preprocessor directive prevents a header file from being included more than once?

A) #once
B) #single
C) #pragma once or #ifndef guard
D) #norepeat

■ **Correct Answer: C) #pragma once or #ifndef guard**
Explanation: Include guards (#ifndef HEADER_H / #define HEADER_H / ... / #endif) or #pragma once prevent multiple inclusion. Without them, redefinition errors can occur when a header is included from multiple files.

Q29. 29. What is an include guard in a header file?

A) A security check on file access
B) A set of preprocessor directives preventing multiple inclusion
C) A function that validates includes
D) A compiler flag

■ **Correct Answer: B) A set of preprocessor directives preventing multiple inclusion**
Explanation: An include guard uses #ifndef SYMBOL / #define SYMBOL / ... / #endif. The first time the header is included, SYMBOL is undefined so the body is processed and SYMBOL is defined. Subsequent inclusions find SYMBOL defined and skip the body.

Q30. 30. Which of the following is a correct include guard structure?

A) #ifdef MYHEADER_H ... #endif
B) #ifndef MYHEADER_H\n#define MYHEADER_H\n...\n#endif
C) #if MYHEADER_H ... #endif
D) #define MYHEADER_H ... #undefine MYHEADER_H

■ **Correct Answer: B) #ifndef MYHEADER_H\n#define MYHEADER_H\n...\n#endif**
Explanation: #ifndef checks if the macro is NOT defined. If not defined, it defines the macro and includes the header content. #endif closes the conditional block. This is the standard include guard pattern.

Q31. 31. What can typically be placed inside a user-defined header file?

A) Only variable declarations
B) Only function definitions
C) Function prototypes, macros, structure definitions, and type definitions
D) Only executable code

■ **Correct Answer: C) Function prototypes, macros, structure definitions, and type definitions**
Explanation: Header files typically contain: function prototypes (declarations), macro definitions (#define), structure/union/enum definitions, typedef declarations, extern variable declarations, and inline function definitions.

Q32. 32. What is the main purpose of a header file in C?

A) To store executable code
B) To provide type safety at runtime
C) To declare interfaces shared across multiple source files
D) To optimize program speed

■ **Correct Answer: C) To declare interfaces shared across multiple source files**
Explanation: Header files serve as the 'interface' to a module. They declare what is available (functions, types, macros) without revealing the implementation details, enabling modular programming and code reuse.

Q33. 33. If a user-defined header 'mymath.h' is in the same directory as the source file, which include is correct?

A) #include
B) #include "../mymath.h"
C) #include "mymath.h"
D) #include {mymath.h}

■ **Correct Answer: C) #include "mymath.h"**
Explanation: #include "mymath.h" searches first in the current directory (where the source file resides), making it the correct syntax when the header is in the same directory as the .c file.

Q34. 34. Placing function definitions (not prototypes) in a header file that is included by multiple .c files can cause:

A) Faster compilation
B) Multiple definition errors at link time
C) Better code reuse
D) No issues

■ **Correct Answer: B) Multiple definition errors at link time**
Explanation: If a function body is in a header and that header is included by multiple .c files, each .c file gets its own copy of the function. The linker sees multiple definitions of the same function and raises an error (unless declared inline or static).

Q35. 35. The #pragma once directive serves the same purpose as:

A) #define once
B) Include guards (#ifndef/#define/#endif)
C) #include once
D) #undef

■ **Correct Answer: B) Include guards (#ifndef/#define/#endif)**
Explanation: #pragma once is a non-standard but widely supported alternative to include guards. It instructs the compiler to include the file only once per compilation unit, preventing multiple inclusions.

Q36. 36. In which file should the actual implementation (body) of functions be written?

A) In the .h file
B) In the .obj file
C) In the .c source file
D) In the .lib file

■ **Correct Answer: C) In the .c source file**
Explanation: Function implementations (bodies) belong in .c source files. Header files (.h) contain only declarations (prototypes). This separation enables separate compilation and reuse of object files.

Q37. 37. Can a user-defined header file include another header file?

A) No, only source files can include headers
B) Yes, a header can include other headers
C) Only standard headers can include other headers
D) Only with compiler flags

■ **Correct Answer: B) Yes, a header can include other headers**
Explanation: Yes. Headers can include other headers. For example, your myutils.h might #include so that any file including myutils.h automatically gets stdio.h. Include guards prevent any loops or duplication.

Q38. 38. Which keyword is used to declare a variable in a header that is defined in a .c file?

A) global
B) shared
C) extern
D) public

■ **Correct Answer: C) extern**
Explanation: The extern keyword tells the compiler 'this variable exists, but is defined elsewhere.' By putting extern int count; in the header, multiple files can reference the same global variable defined once in a .c file.

Q39. 39. What happens when the preprocessor encounters #include "config.h"?

A) It links config.h at runtime
B) It replaces the directive with the entire contents of config.h
C) It calls config.h as a function
D) It imports config.h as a module

■ **Correct Answer: B) It replaces the directive with the entire contents of config.h**
Explanation: The preprocessor literally replaces the #include directive with the complete text content of the specified file before compilation begins. This is purely textual substitution.

Q40. 40. A header file used across a large project should be placed in:

A) The same directory as main.c only
B) A dedicated include/ or headers/ directory accessible project-wide
C) The /tmp directory
D) Inside the .c source files

■ **Correct Answer: B) A dedicated include/ or headers/ directory accessible project-wide**
Explanation: In large projects, headers are placed in a dedicated include/ directory. The compiler's include path (-I flag in GCC) is set to point to this directory, allowing all source files to access shared headers.

Q41. 41. Which of the following correctly defines a constant PI in a user-defined header?

A) const PI = 3.14159;
B) #define PI 3.14159
C) variable PI = 3.14159;
D) float PI: 3.14159;

■ **Correct Answer: B) #define PI 3.14159**
Explanation: #define PI 3.14159 creates a preprocessor macro. When the code is compiled, all occurrences of PI in the source are textually replaced by 3.14159 before the compiler sees the code.

Q42. 42. Which of the following is NOT appropriate to place in a header file?

A) Function prototypes
B) Struct definitions
C) Non-inline function definitions with large bodies
D) Macro definitions

■ **Correct Answer: C) Non-inline function definitions with large bodies**
Explanation: Non-inline function definitions in headers cause multiple definition errors (one per translation unit that includes the header). Only inline functions, macros, type definitions, and extern declarations should go in headers.

Q43. 43. If two header files each define a struct with the same name and both are included, what occurs?

A) The second definition overwrites the first
B) A redefinition error at compile time
C) The structs are merged
D) No error; both are used

■ **Correct Answer: B) A redefinition error at compile time**
Explanation: C does not allow redefinition of a struct with the same tag in the same translation unit. Including two headers that each define struct Point { }; causes a compile-time 'redefinition' error. Include guards or namespacing the struct names prevent this.

Q44. 44. The convention of naming include guards using the header filename in uppercase with underscores (e.g., MYMATH_H) is:

A) Mandatory by the C standard
B) A widely accepted best practice to avoid naming conflicts
C) Only used in C++
D) Used only in embedded systems

■ **Correct Answer: B) A widely accepted best practice to avoid naming conflicts**
Explanation: Using FILENAME_H_ or _FILENAME_H as include guard macros is a widely followed convention. The C standard reserves identifiers starting with underscore for the implementation, so MYMATH_H (without leading underscore) is the common safe choice.

Q45. 45. Which of the following is the correct syntax to define a user-defined function prototype in a header?

A) function int add(int, int);
B) prototype int add(int, int);
C) int add(int a, int b);
D) declare int add(int, int);

■ **Correct Answer: C) int add(int a, int b);**
Explanation: A function prototype in C is simply the function signature followed by a semicolon: int add(int a, int b); Parameter names are optional in prototypes. The extern keyword is implied for function declarations.

Q46. 46. Including a header file with angle brackets vs double quotes differs in:

A) The file encoding used
B) The search path order the preprocessor uses
C) Whether the file is compiled
D) The link libraries used

■ **Correct Answer: B) The search path order the preprocessor uses**
Explanation: Angle brackets <> tell the preprocessor to search standard/system include directories only. Double quotes "" tell it to first search the current directory (and user-specified -I paths), then fall back to system directories.

Q47. 47. Which of the following is a valid user-defined header filename?

A) my file.h (with space)
B) my-utils.h
C) my_utils.h
D) Both B and C

■ **Correct Answer: D) Both B and C**
Explanation: Both my-utils.h and my_utils.h are valid filenames. Spaces in filenames cause issues with include directives and are strongly discouraged. Underscores and hyphens are both acceptable, though underscores (my_utils.h) are more conventional in C.

Q48. 48. What is a forward declaration in the context of header files?

A) Including a header before main()
B) Declaring an identifier before it is fully defined
C) Defining a function before all variables
D) A type of preprocessor macro

■ **Correct Answer: B) Declaring an identifier before it is fully defined**
Explanation: A forward declaration announces the existence of a function or type without providing its full definition. For example, struct Node; declares that struct Node exists, allowing pointers to it before the full struct definition.

Q49. 49. In a project, config.h is included in 5 different .c files. Without include guards, this causes:

A) Faster compilation
B) Exactly one copy of definitions
C) Possible multiple redefinition errors
D) No problem

■ **Correct Answer: C) Possible multiple redefinition errors**
Explanation: Without include guards, each #include "config.h" in each .c file inserts the full content. If any two source files both include a header that defines a struct or extern, the linker/compiler may see duplicate definitions and report errors.

Q50. 50. Which of these correctly creates a user-defined header file for a simple calculator module?

A) calc.h with full function code and local variables

B) calc.h with #include guards, prototypes (add, sub, mul, div), and any needed #defines

C) calc.h with only a main() function

D) calc.h with #include

■ **Correct Answer: B) calc.h with #include guards, prototypes (add, sub, mul, div), and any needed #defines**
Explanation: A proper calculator header (calc.h) should contain: include guards, function prototypes (int add(int,int); etc.), any relevant macros (#define MAX_INPUT 1000), and possibly struct definitions — but NOT function bodies or local variable definitions.

Section 3: Uses of Header Files & Library Functions
Questions 51–100

Q51. 51. What is the output of strlen("HELLO")?

A) 6
B) 5
C) 4
D) 0

■ **Correct Answer: B) 5**
Explanation: strlen() counts characters excluding the null terminator '\0'. "HELLO" has 5 characters (H,E,L,L,O), so strlen() returns 5. The string is stored as 6 bytes including '\0', but strlen reports 5.

Q52. 52. Which function copies a string, including the null terminator?

A) strcat()
B) strcmp()
C) strcpy()
D) strlen()

■ **Correct Answer: C) strcpy()**
Explanation: strcpy(dest, src) copies the null-terminated string src into dest, including the terminating '\0'. It returns dest. Care must be taken that dest has sufficient space; otherwise buffer overflow occurs.

Q53. 53. The function strcat(dest, src) does what?

A) Compares dest and src
B) Copies src to dest
C) Appends src to the end of dest
D) Returns the length of dest+src

■ **Correct Answer: C) Appends src to the end of dest**
Explanation: strcat() appends the string src (including its '\0') to the end of dest (overwriting dest's '\0'). It returns dest. The destination must have enough space for both strings plus the null terminator.

Q54. 54. What does strcmp("abc", "abc") return?

A) 1
B) -1
C) 0
D) Length of string

■ **Correct Answer: C) 0**
Explanation: strcmp() returns 0 when both strings are identical, a negative value when the first string is lexicographically less than the second, and a positive value when it is greater.

Q55. 55. Which function in stdio.h reads a formatted string from stdin?

A) gets()
B) puts()
C) scanf()
D) printf()

■ **Correct Answer: C) scanf()**
Explanation: scanf() reads formatted input from stdin according to a format string. %d reads integers, %f reads floats, %s reads strings, %c reads characters. It returns the number of items successfully read.

Q56. 56. What does fopen("file.txt", "r") return if the file does not exist?

A) An empty FILE pointer
B) NULL
C) 0
D) EOF

■ **Correct Answer: B) NULL**
Explanation: fopen() returns NULL when the file cannot be opened (file not found, permission denied etc.). Always check if fopen() returns NULL before using the file pointer to avoid undefined behavior.

Q57. 57. The fclose() function is used to:

A) Delete a file
B) Move file pointer to beginning
C) Close an open file and flush its buffer
D) Check end of file

■ **Correct Answer: C) Close an open file and flush its buffer**
Explanation: fclose() flushes any buffered output, closes the file, and releases the FILE structure. It returns 0 on success and EOF on error. Always call fclose() after file operations to prevent resource leaks.

Q58. 58. Which function checks if the end of file has been reached?

A) eof()
B) fend()
C) feof()
D) endfile()

■ **Correct Answer: C) feof()**
Explanation: feof(fp) returns non-zero (true) when the end-of-file indicator for stream fp has been set, i.e., a previous read operation reached the end of the file.

Q59. 59. The function atoi() from stdlib.h converts:

A) Integer to ASCII
B) ASCII string to integer
C) Integer to float
D) Array to integer

■ **Correct Answer: B) ASCII string to integer**
Explanation: atoi() (ASCII to Integer) converts a string representation of an integer to an int. For example, atoi("42") returns 42. For better error handling, strtol() is preferred in modern code.

Q60. 60. Which math.h function returns the absolute value of a floating-point number?

A) abs()
B) absolute()
C) fabs()
D) modulus()

■ **Correct Answer: C) fabs()**
Explanation: fabs() returns the absolute value of a double. For integers, abs() from stdlib.h is used. fabsf() works with float and fabsl() with long double. Using abs() on a float may give incorrect results.

Q61. 61. The ceil() function in math.h returns:

A) The floor of a number
B) The square root
C) The smallest integer not less than x
D) The rounded value

■ **Correct Answer: C) The smallest integer not less than x**
Explanation: ceil(x) returns the smallest integer value that is greater than or equal to x (rounds up). For example, ceil(2.3) = 3.0, ceil(-2.3) = -2.0. It returns a double.

Q62. 62. What does floor(3.7) return?

A) 4.0
B) 3.0
C) 3.7
D) 4

■ **Correct Answer: B) 3.0**
Explanation: floor(x) returns the largest integer value not greater than x (rounds down toward negative infinity). floor(3.7) = 3.0, floor(-3.7) = -4.0. Return type is double.

Q63. 63. Which function from stdlib.h allocates memory for an array of elements, initializing them to zero?

A) malloc()
B) realloc()
C) alloc()
D) calloc()

■ **Correct Answer: D) calloc()**
Explanation: calloc(n, size) allocates memory for n elements each of given size bytes and initializes ALL bytes to zero. malloc() allocates but does NOT initialize. calloc(10, sizeof(int)) allocates space for 10 integers, all set to 0.

Q64. 64. The realloc(ptr, new_size) function:

A) Allocates new memory and frees old automatically
B) Only increases the size of allocated memory
C) Resizes previously allocated memory, possibly moving it
D) Copies old data then frees it manually

■ **Correct Answer: C) Resizes previously allocated memory, possibly moving it**
Explanation: realloc() attempts to resize the memory block at ptr to new_size bytes. It may return the same pointer (if resized in-place) or a new pointer (if moved). The old pointer becomes invalid if a new address is returned.

Q65. 65. What does free(ptr) do when ptr is NULL?

A) Crashes the program
B) Generates a compile error
C) Does nothing (safe to call)
D) Frees all heap memory

■ **Correct Answer: C) Does nothing (safe to call)**
Explanation: The C standard guarantees that free(NULL) is a no-op (does nothing). This makes it safe to call free on a pointer that may or may not have been allocated, without an explicit NULL check.

Q66. 66. Which function converts an integer to a string representation in C?

A) itoa() or sprintf()
B) strtoi()
C) inttostr()
D) tostring()

■ **Correct Answer: A) itoa() or sprintf()**
Explanation: itoa() is a non-standard function (available in some compilers). The standard approach is sprintf(buffer, "%d", number) which writes the integer as a string into buffer. snprintf() is safer as it limits the output size.

Q67. 67. The toupper() function from ctype.h:

A) Converts a string to uppercase
B) Converts a single character to uppercase
C) Checks if character is uppercase
D) Returns the ASCII value of uppercase

■ **Correct Answer: B) Converts a single character to uppercase**
Explanation: toupper(c) converts a single lowercase character c to its uppercase equivalent. If c is not a lowercase letter, it is returned unchanged. To convert a string, loop through each character and apply toupper().

Q68. 68. Which function from stdio.h writes a character to a file?

A) fputchar()
B) putc() or fputc()
C) writechar()
D) charout()

■ **Correct Answer: B) putc() or fputc()**
Explanation: fputc(c, fp) writes the character c to the file stream fp. putc(c, fp) is typically a macro equivalent of fputc(). Both return the character written on success or EOF on error.

Q69. 69. The fgets(str, n, fp) function reads:

A) n characters and always adds '\0'
B) Up to n-1 characters from fp, adding '\n' and '\0'
C) Exactly n characters without null terminator
D) A full line ignoring the limit n

■ **Correct Answer: B) Up to n-1 characters from fp, adding '\n' and '\0'**
Explanation: fgets() reads at most n-1 characters from stream fp into str, stopping at newline (which IS stored in str) or EOF, then appends '\0'. It is safer than gets() which has no buffer size limit.

Q70. 70. What does putchar(c) do?

A) Puts a string to stdout
B) Writes character c to stdout
C) Reads a character from stdin
D) Writes c to a file

■ **Correct Answer: B) Writes character c to stdout**
Explanation: putchar(c) is equivalent to fputc(c, stdout). It writes a single character to standard output. It is declared in stdio.h and returns the character written on success or EOF on failure.

Q71. 71. The getchar() function:

A) Gets a string from stdin
B) Gets a character from a file
C) Gets a single character from stdin
D) Gets an integer from stdin

■ **Correct Answer: C) Gets a single character from stdin**
Explanation: getchar() reads the next character from standard input (stdin). It is equivalent to fgetc(stdin). It returns the character as an unsigned char converted to int, or EOF at end-of-file.

Q72. 72. Which function uses a format string to write to a character array (string)?

A) fprintf()
B) printf()
C) sprintf()
D) sscanf()

■ **Correct Answer: C) sprintf()**
Explanation: sprintf(buffer, format, ...) works like printf() but writes the formatted output into the character array buffer instead of stdout. snprintf(buffer, size, format, ...) is safer as it limits output to size characters.

Q73. 73. The rand() function from stdlib.h returns:

A) A true random number
B) A pseudo-random integer in [0, RAND_MAX]
C) A float between 0 and 1
D) A random character

■ **Correct Answer: B) A pseudo-random integer in [0, RAND_MAX]**
Explanation: rand() returns a pseudo-random integer between 0 and RAND_MAX (at least 32767). The sequence is deterministic; seed it with srand(time(NULL)) for different sequences each run.

Q74. 74. Which function from time.h returns the current calendar time?

A) clock()
B) gettime()
C) time()
D) now()

■ **Correct Answer: C) time()**
Explanation: time(NULL) returns the current time as a time_t value (typically seconds since the Unix epoch, Jan 1 1970). clock() returns CPU time used, not wall-clock time.

Q75. 75. The memset(ptr, value, n) function sets:

A) n integers starting at ptr to value
B) n bytes starting at ptr to value
C) All memory to value
D) The pointer ptr to value

■ **Correct Answer: B) n bytes starting at ptr to value**
Explanation: memset() sets the first n bytes of the memory area pointed to by ptr to the byte value (as unsigned char). It is commonly used to initialize arrays/buffers: memset(arr, 0, sizeof(arr)) zeros the array.

Q76. 76. Which function searches for the first occurrence of a character in a string?

A) strstr()
B) strchr()
C) strfind()
D) strsearch()

■ **Correct Answer: B) strchr()**
Explanation: strchr(str, c) returns a pointer to the first occurrence of character c in string str, or NULL if not found. strrchr() finds the LAST occurrence. strstr() searches for a substring, not a single character.

Q77. 77. The pow(x, y) function computes:

A) x + y
B) x * y
C) x raised to the power y
D) The y-th root of x

■ **Correct Answer: C) x raised to the power y**
Explanation: pow(x, y) returns x^y (x raised to the power y) as a double. pow(2.0, 10.0) = 1024.0. Both arguments must be double. For integer powers, explicit multiplication in a loop avoids floating-point inaccuracies.

Q78. 78. Which header provides qsort() for sorting arrays?

A) sort.h
B) search.h
C) stdlib.h
D) algorithm.h

■ **Correct Answer: C) stdlib.h**
Explanation: qsort() is declared in stdlib.h. It sorts an array using a comparison function provided by the programmer: qsort(arr, n, sizeof(int), compare). The comparison function returns negative, zero, or positive values.

Q79. 79. The bsearch() function in stdlib.h:

A) Performs bubble sort
B) Does binary search on a sorted array
C) Computes a checksum
D) Finds the largest element

■ **Correct Answer: B) Does binary search on a sorted array**
Explanation: bsearch() performs binary search on a sorted array. Like qsort(), it takes a comparison function. It returns a pointer to the matching element or NULL if not found. The array MUST be sorted first.

Q80. 80. What does isspace(' ') return?

A) 0
B) -1
C) Non-zero (true)
D) The ASCII value of space

■ **Correct Answer: C) Non-zero (true)**
Explanation: isspace(c) from ctype.h returns non-zero (true) if c is a whitespace character: space ' ', tab '\t', newline '\n', carriage return '\r', form feed '\f', or vertical tab '\v'. isspace(' ') returns non-zero.

Q81. 81. Which stdio.h function repositions the file pointer to the beginning of a file?

A) fseek(fp, 0, SEEK_END)
B) ftell(fp)
C) rewind(fp)
D) fflush(fp)

■ **Correct Answer: C) rewind(fp)**
Explanation: rewind(fp) moves the file position indicator to the beginning (offset 0) of the file stream fp and clears the end-of-file and error indicators. It is equivalent to fseek(fp, 0L, SEEK_SET).

Q82. 82. The fseek(fp, offset, whence) function with SEEK_CUR as whence moves the file pointer:

A) To the beginning of file
B) To the end of file
C) Relative to the current position
D) To an absolute byte position

■ **Correct Answer: C) Relative to the current position**
Explanation: SEEK_CUR moves the file pointer offset bytes relative to the current position. SEEK_SET moves to an absolute position from the file beginning. SEEK_END moves relative to the end of file.

Q83. 83. What does ftell(fp) return?

A) Total file size in bytes
B) Current position of the file pointer as a long int
C) Number of bytes remaining
D) The last character read

■ **Correct Answer: B) Current position of the file pointer as a long int**
Explanation: ftell() returns the current value of the position indicator for stream fp as a long int (byte offset from the beginning). It returns -1L on error. Combined with fseek(), it is used to determine file size.

Q84. 84. The perror(str) function from stdio.h:

A) Prints str to a file
B) Prints a system error message corresponding to errno
C) Generates a runtime error
D) Checks for programming errors

■ **Correct Answer: B) Prints a system error message corresponding to errno**
Explanation: perror(str) prints str followed by a colon and the error message corresponding to the current value of errno (set by library functions on error). For example, after a failed fopen(), perror("Error") might print 'Error: No such file or directory'.

Q85. 85. Which function from string.h compares two strings case-insensitively?

A) strcmp()
B) stricmp() / strcasecmp()
C) strncmp()
D) strcoll()

■ **Correct Answer: B) stricmp() / strcasecmp()**
Explanation: strcasecmp() (POSIX/Linux) or stricmp() (Windows/Borland) compare strings ignoring case. The standard strcmp() is case-sensitive. strcoll() compares using locale rules. strncmp() compares at most n characters.

Q86. 86. The strtok(str, delimiters) function is used for:

A) Converting string to token value
B) Splitting a string into tokens using delimiter characters
C) Counting tokens in a string
D) Encrypting a string

■ **Correct Answer: B) Splitting a string into tokens using delimiter characters**
Explanation: strtok() tokenizes a string by replacing delimiter characters with '\0' and returning pointers to each token. First call: strtok(str, delim). Subsequent calls: strtok(NULL, delim). It modifies the original string.

Q87. 87. The return type of malloc() is:

A) int*
B) char*
C) void*
D) void

■ **Correct Answer: C) void***
Explanation: malloc() returns void* (a generic pointer). In C, void* is automatically converted to any pointer type upon assignment, so explicit casting is not required (unlike C++). If allocation fails, it returns NULL.

Q88. 88. Which function from math.h computes the natural logarithm?

A) log10()
B) log()
C) ln()
D) loge()

■ **Correct Answer: B) log()**
Explanation: log(x) computes the natural logarithm (base e) of x. log10(x) computes the base-10 logarithm. log2(x) (C99) computes the base-2 logarithm. All return double; x must be positive.

Q89. 89. What is the purpose of fflush(stdout)?

A) Closes stdout
B) Forces all buffered output for stdout to be written immediately
C) Reads remaining input from stdout
D) Resets the stdout format

■ **Correct Answer: B) Forces all buffered output for stdout to be written immediately**
Explanation: fflush() forces any buffered data for the given stream to be written to its underlying file or device. fflush(stdout) ensures pending printf output appears immediately, which is useful before reading input or debugging.

Q90. 90. The abs() function in stdlib.h works on which data type?

A) double
B) float
C) int
D) long double

■ **Correct Answer: C) int**
Explanation: abs() in stdlib.h operates on int values. For long integers, use labs(). For long long, use llabs() (C99). For floating-point, use fabs() (for double), fabsf() (for float), fabsl() (for long double) from math.h.

Q91. 91. Which header provides the system() function to execute OS commands?

A) os.h
B) system.h
C) stdlib.h
D) shell.h

■ **Correct Answer: C) stdlib.h**
Explanation: system(command) from stdlib.h passes command to the host OS shell for execution (e.g., system("ls") on Linux, system("dir") on Windows). It returns the command's exit status.

Q92. 92. The localtime() function from time.h returns a pointer to:

A) A string representation of time
B) A struct tm containing broken-down local time
C) The GMT time as time_t
D) A calendar page number

■ **Correct Answer: B) A struct tm containing broken-down local time**
Explanation: localtime(time_t*) converts a time_t value to a struct tm representing local time with fields: tm_year (years since 1900), tm_mon (0-11), tm_mday (1-31), tm_hour, tm_min, tm_sec etc.

Q93. 93. Which function from string.h finds the first occurrence of a substring?

A) strchr()
B) strpos()
C) strstr()
D) substr()

■ **Correct Answer: C) strstr()**
Explanation: strstr(haystack, needle) returns a pointer to the first occurrence of string needle within string haystack, or NULL if not found. It searches for the entire substring, unlike strchr() which searches for a single character.

Q94. 94. What is RAND_MAX?

A) The maximum size of an array
B) The maximum value rand() can return
C) The maximum random seed value
D) A function in math.h

■ **Correct Answer: B) The maximum value rand() can return**
Explanation: RAND_MAX is a macro defined in stdlib.h that represents the maximum value that rand() can return. The C standard guarantees it is at least 32767. The actual value is implementation-specific (often 32767 or 2147483647).

Q95. 95. Which function safely limits the number of characters copied to prevent buffer overflow?

A) strcpy()
B) strncpy()
C) strcat()
D) strcmp()

■ **Correct Answer: B) strncpy()**
Explanation: strncpy(dest, src, n) copies at most n characters from src to dest. Unlike strcpy(), it does not overflow the buffer. However, if src has n or more characters, strncpy() may NOT null-terminate dest, requiring explicit handling.

Q96. 96. The difftime(t2, t1) function returns:

A) The current time
B) The difference t2-t1 in seconds as a double
C) A formatted date string
D) The number of clock ticks between t1 and t2

■ **Correct Answer: B) The difference t2-t1 in seconds as a double**
Explanation: difftime(time2, time1) computes the difference between two time_t values and returns it as a double representing the number of seconds elapsed from time1 to time2.

Q97. 97. Which ctype.h function checks if a character is a decimal digit?

A) isalpha()
B) isnum()
C) isnumeric()
D) isdigit()

■ **Correct Answer: D) isdigit()**
Explanation: isdigit(c) returns non-zero if c is a decimal digit character ('0' through '9'). isalpha() checks for letters, isalnum() checks for letters or digits, isxdigit() checks for hexadecimal digits.

Q98. 98. The function strrev() that reverses a string is:

A) Standard C (ANSI)
B) C99 standard
C) Non-standard (available in some compilers like Borland)
D) Part of string.h in all compilers

■ **Correct Answer: C) Non-standard (available in some compilers like Borland)**
Explanation: strrev() is NOT part of the ANSI/ISO C standard. It is available in Borland C/Turbo C. To reverse a string in standard C, implement it manually using a loop or use other string functions.

Q99. 99. Which function writes formatted data to a file stream?

A) printf()
B) sprintf()
C) fprintf()
D) dprintf()

■ **Correct Answer: C) fprintf()**
Explanation: fprintf(fp, format, ...) works like printf() but writes to the specified file stream fp instead of stdout. fprintf(stderr, ...) is commonly used to write error messages to standard error.

Q100. 100. What does memcpy(dest, src, n) do?

A) Compares n bytes of dest and src
B) Moves n bytes from src to dest (handles overlap)
C) Copies n bytes from src to dest (undefined behavior if overlapping)
D) Sets n bytes of dest to zero

■ **Correct Answer: C) Copies n bytes from src to dest (undefined behavior if overlapping)**
Explanation: memcpy() copies exactly n bytes from the memory area src to dest. If the source and destination overlap, behavior is undefined — use memmove() instead, which correctly handles overlapping regions.

Section 4: Advanced & Conceptual Questions *Questions 101–125*

Q101. 101. The #include directive is processed by the:

A) Compiler
B) Linker
C) Preprocessor
D) Assembler

■ **Correct Answer: C) Preprocessor**
Explanation: The C preprocessor runs before the compiler. It handles all directives starting with #: #include (file inclusion), #define (macros), #ifdef/#ifndef (conditional compilation), #pragma (compiler-specific instructions).

Q102. 102. Which of the following is true about header files and compilation?

A) Header files are compiled separately and linked
B) Header files are textually inserted into source files before compilation
C) Header files produce object files
D) Header files are interpreted at runtime

■ **Correct Answer: B) Header files are textually inserted into source files before compilation**
Explanation: The preprocessor replaces each #include directive with the exact text contents of the specified file. The compiler then compiles the expanded source (called a translation unit) as a single unit. Header files do NOT produce object files.

Q103. 103. What is a translation unit in C?

A) A compiled library
B) A single .c file after preprocessor expansion (including all headers)
C) A .h file with its definitions
D) A group of .c files compiled together

■ **Correct Answer: B) A single .c file after preprocessor expansion (including all headers)**
Explanation: A translation unit is the source code seen by the compiler: a single .c source file plus all the headers it includes (after preprocessing). Each translation unit is compiled independently into an object file (.o).

Q104. 104. The One Definition Rule (ODR) in C states:

A) Every variable must be defined exactly once across all translation units
B) Only one header can be included per .c file
C) Functions can be defined multiple times
D) A program can have only one source file

■ **Correct Answer: A) Every variable must be defined exactly once across all translation units**
Explanation: Each global variable and non-inline function must have exactly ONE definition across all translation units. Multiple declarations (with extern) are allowed, but multiple definitions cause linker errors. This is the One Definition Rule.

Q105. 105. Which of the following correctly represents the order of C compilation stages?

A) Preprocessing → Compilation → Assembly → Linking
B) Compilation → Preprocessing → Linking → Assembly
C) Assembly → Compilation → Preprocessing → Linking
D) Preprocessing → Linking → Compilation → Assembly

■ **Correct Answer: A) Preprocessing → Compilation → Assembly → Linking**
Explanation: The standard C compilation pipeline: (1) Preprocessing — handles #include, #define, #ifdef; (2) Compilation — translates C to assembly; (3) Assembly — converts assembly to machine code (object file); (4) Linking — combines object files and libraries into an executable.

Q106. 106. An inline function defined in a header file:

A) Cannot be in a header file
B) Violates the One Definition Rule
C) Is allowed in headers; each translation unit gets its own copy
D) Must be declared extern

■ **Correct Answer: C) Is allowed in headers; each translation unit gets its own copy**
Explanation: The inline keyword is an exception to ODR. Inline function definitions may appear in multiple translation units (each sees the same definition via the header). The compiler may substitute the function body at the call site to avoid function call overhead.

Q107. 107. Which header file contains the macro offsetof(type, member)?

A) stdio.h
B) stdlib.h
C) stddef.h
D) string.h

■ **Correct Answer: C) stddef.h**
Explanation: offsetof(type, member) is a macro in stddef.h that returns the byte offset of member within a struct or union type. It is extensively used in container_of and similar patterns in systems programming.

Q108. 108. When you use gcc -E file.c, what does gcc produce?

A) The final executable
B) An object file (.o)
C) The preprocessed output (C code after #includes expansion)
D) Assembly code

■ **Correct Answer: C) The preprocessed output (C code after #includes expansion)**
Explanation: gcc -E stops after the preprocessing stage and prints the preprocessed source to stdout. This is useful for debugging: you can see exactly what text the compiler receives after all #include and #define expansions.

Q109. 109. The C standard header iso646.h provides:

A) ISO country codes
B) Alternative spellings of logical/bitwise operators
C) Character encoding mappings
D) Integer overflow macros

■ **Correct Answer: B) Alternative spellings of logical/bitwise operators**
Explanation: iso646.h (C94) defines alternative tokens for operators: and (&&), or (||), not (!), xor (^), bitand (&), bitor (|), and_eq (&=), or_eq (|=), xor_eq (^=), not_eq (!=). Useful where keyboard characters are limited.

Q110. 110. #pragma once is:

A) Defined by the ANSI C standard
B) A C99 mandatory feature
C) A non-standard but widely supported compiler extension
D) Defined in stddef.h

■ **Correct Answer: C) A non-standard but widely supported compiler extension**
Explanation: #pragma once is not part of the C standard (which defines no mandatory #pragmas), but is supported by GCC, Clang, MSVC and most modern compilers. It is simpler than include guards but not 100% portable to obscure compilers.

Q111. 111. The __FILE__ and __LINE__ predefined macros are declared in:

A) stdio.h
B) debug.h
C) They are compiler-predefined, not from any header
D) stddef.h

■ **Correct Answer: C) They are compiler-predefined, not from any header**
Explanation: __FILE__ (current filename as string), __LINE__ (current line number as int), __DATE__, __TIME__, and __func__ (C99, current function name) are predefined macros built into the C preprocessor — no header needed.

Q112. 112. Which header file should be included to use POSIX threads (pthreads)?

A) thread.h
B) pthread.h
C) threads.h (C11)
D) Both B (POSIX) and C (C11 standard)

■ **Correct Answer: D) Both B (POSIX) and C (C11 standard)**
Explanation: POSIX systems use pthread.h (#include) with -lpthread linking. The C11 standard introduced its own threading in threads.h (). Both exist but serve slightly different contexts.

Q113. 113. In C, the header file extension .h is:

A) Mandatory, enforced by the compiler
B) A convention; any extension works technically
C) Required only for standard headers
D) Used only for user-defined headers

■ **Correct Answer: B) A convention; any extension works technically**
Explanation: The .h extension is a widely followed convention but NOT enforced by the language. The preprocessor doesn't care about the extension — it simply inserts the file contents. However, deviating from .h causes confusion and should be avoided.

Q114. 114. What is the purpose of the extern "C" declaration in a header file?

A) It allows the header to be used in both C and C++
B) It declares external OS functions
C) It prevents the header from compiling in C
D) It links the header to an external library

■ **Correct Answer: A) It allows the header to be used in both C and C++**
Explanation: In C++ code, functions in a C header need extern "C" to prevent C++ name mangling, ensuring proper linkage with C-compiled object files. This is typically wrapped in #ifdef __cplusplus so the header works in both C and C++.

Q115. 115. The tgmath.h header (C99) provides:

A) Trigonometric and geometric math
B) Type-generic math macros that select correct math functions based on argument type
C) Integer arithmetic only
D) A replacement for math.h

■ **Correct Answer: B) Type-generic math macros that select correct math functions based on argument type**
Explanation: tgmath.h provides type-generic macros (using _Generic in C11, or compiler extensions in C99) that automatically select the appropriate math function variant: sqrt() vs sqrtf() vs sqrtl() based on the argument type.

Q116. 116. Which of the following header files provides support for atomic operations (C11)?

A) atomic.h
B) sync.h
C) stdatomic.h
D) thread.h

■ **Correct Answer: C) stdatomic.h**
Explanation: stdatomic.h (C11) provides atomic types (atomic_int, atomic_bool etc.) and atomic operations (atomic_load, atomic_store, atomic_fetch_add etc.) for lock-free thread-safe programming.

Q117. 117. The function snprintf(buf, size, format, ...) compared to sprintf() is:

A) Slower
B) Safer — it limits output to size-1 characters, preventing buffer overflow
C) Only works with integers
D) Does not support format strings

■ **Correct Answer: B) Safer — it limits output to size-1 characters, preventing buffer overflow**
Explanation: snprintf() is the safe version of sprintf(). It writes at most size-1 characters into buf and always null-terminates. sprintf() has no size limit and can cause buffer overflows — snprintf() is strongly preferred in modern C.

Q118. 118. When multiple header files are included, the order of inclusion:

A) Does not matter at all
B) Can matter if one header depends on types or macros defined in another
C) Is alphabetical by convention
D) Is determined by the linker

■ **Correct Answer: B) Can matter if one header depends on types or macros defined in another**
Explanation: Include order can matter. If mylib.h uses types from stdint.h, then stdint.h must be included first (or mylib.h should include stdint.h itself). Well-designed headers are self-contained — they include all their own dependencies.

Q119. 119. The _Generic keyword introduced in C11 is used in headers to:

A) Generate code at runtime
B) Create type-generic macros that select different expressions based on type
C) Define generic structures
D) Import generic libraries

■ **Correct Answer: B) Create type-generic macros that select different expressions based on type**
Explanation: _Generic(expr, type1: val1, type2: val2, ...) is a C11 compile-time selection expression. It enables type-generic macros in headers, used in tgmath.h to select sqrt(), sqrtf(), or sqrtl() based on argument type.

Q120. 120. A header file should be self-contained, meaning:

A) It should not include any other headers
B) It compiles correctly if included first, without requiring prior includes
C) It contains complete function bodies
D) It has no dependencies on other files

■ **Correct Answer: B) It compiles correctly if included first, without requiring prior includes**
Explanation: A self-contained header includes all the headers it depends on. If mylib.h uses size_t, it should #include itself, not rely on the user having already included it. This prevents subtle errors and ordering problems.

Q121. 121. The MATCH: Which header goes with each function?
(i) printf — (ii) malloc — (iii) strlen — (iv) sqrt

A) stdio.h, stdlib.h, string.h, math.h
B) stdio.h, string.h, stdlib.h, math.h
C) stdlib.h, stdio.h, math.h, string.h
D) math.h, stdlib.h, stdio.h, string.h

■ **Correct Answer: A) stdio.h, stdlib.h, string.h, math.h**
Explanation: printf() → stdio.h (Standard I/O); malloc() → stdlib.h (Standard Library); strlen() → string.h (String operations); sqrt() → math.h (Mathematical functions). This mapping is fundamental knowledge.

Q122. 122. MATCH: Header file with its primary purpose.
(i) ctype.h — (ii) errno.h — (iii) assert.h — (iv) limits.h

A) Character classification, Error codes, Debugging assertions, Integer limits
B) String operations, File errors, Testing, Floating-point limits
C) Math, Error handling, Memory, Strings
D) I/O, Sorting, Assertions, Locale

■ **Correct Answer: A) Character classification, Error codes, Debugging assertions, Integer limits**
Explanation: ctype.h → character classification (isalpha, isdigit); errno.h → error code variable and constants; assert.h → assert() debugging macro; limits.h → integer type range constants (INT_MAX etc.).

Q123. 123. CONCEPTUAL: Why should header files NOT contain global variable definitions (without extern)?

A) Global variables cannot be declared in headers
B) Including the header in multiple .c files creates multiple definitions, causing linker errors
C) Global variables in headers are automatically extern
D) The preprocessor ignores variable definitions

■ **Correct Answer: B) Including the header in multiple .c files creates multiple definitions, causing linker errors**
Explanation: If int count = 0; (a definition) appears in a header included by 3 .c files, each .c file defines its own count. The linker sees 3 definitions of the same global symbol and raises a 'multiple definition' error. Use extern int count; in the header and define it in exactly one .c file.

Q124. 124. CONCEPTUAL: What is the difference between declaration and definition in C?

A) They are the same
B) Declaration introduces an identifier; definition also allocates storage/provides body
C) Definitions are in headers; declarations in .c files
D) Declarations allocate memory; definitions do not

■ **Correct Answer: B) Declaration introduces an identifier; definition also allocates storage/provides body**
Explanation: Declaration: announces a name and its type to the compiler (e.g., extern int x; or int add(int,int);) — no storage allocated. Definition: actually creates the entity (e.g., int x = 5; or a function with its body) — storage is allocated. Headers declare; source files define.

Q125. 125. CONCEPTUAL: Which statement about #include and the preprocessor is CORRECT?

A) #include compiles the header file separately
B) #include statements are resolved at runtime by the OS
C) The preprocessor replaces #include with file contents before compilation
D) #include adds the header to the linker's search path

■ **Correct Answer: C) The preprocessor replaces #include with file contents before compilation**
Explanation: The preprocessor (phase 1 of compilation) handles #include by reading the specified file and replacing the #include line with that file's entire textual content. This expanded source is then passed to the compiler. No separate compilation of headers occurs (unless using precompiled headers).

Appendix A: Quick Reference — Header Files in C

Header File	Key Contents / Functions
<stdio.h>	printf, scanf, fprintf, fscanf, fopen, fclose, fgets, fputs, getchar, putchar, fread, fwrite, fseek, ftell, rewind, feof, perror, sprintf, sscanf
<stdlib.h>	malloc, calloc, realloc, free, exit, abort, atoi, atof, atol, rand, srand, qsort, bsearch, abs, system, getenv, strtol, strtod
<string.h>	strlen, strcpy, strncpy, strcat, strncat, strcmp, strncmp, strchr, strrchr, strstr, strtok, memset, memcpy, memmove, memcmp
<math.h>	sqrt, pow, fabs, floor, ceil, sin, cos, tan, log, log10, exp, sinh, cosh, tanh, asin, acos, atan, atan2, fmod, modf
<ctype.h>	isalpha, isdigit, isalnum, isspace, isupper, islower, isupper, ispunct, isprint, toupper, tolower, isxdigit
<time.h>	time, clock, difftime, mktime, localtime, gmtime, asctime, ctime, strftime — types: time_t, struct tm, clock_t
<assert.h>	assert() macro — disabled when NDEBUG is defined
<errno.h>	errno variable — EDOM, ERANGE, ENOMEM, ENOENT etc.
<signal.h>	signal(), raise() — SIGINT, SIGTERM, SIGSEGV, SIGFPE, SIGABRT
<limits.h>	CHAR_BIT, CHAR_MIN/MAX, SHRT_MIN/MAX, INT_MIN/MAX, LONG_MIN/MAX, UINT_MAX, ULONG_MAX
<float.h>	FLT_MAX/MIN/EPSILON, DBL_MAX/MIN/EPSILON, LDBL_MAX, FLT_DIG, DBL_DIG, FLT_MANT_DIG
<stddef.h>	NULL, size_t, ptrdiff_t, wchar_t, offsetof()
<stdarg.h>	va_list, va_start(), va_arg(), va_end(), va_copy() — variadic functions
<stdint.h>	int8_t, uint8_t, int16_t, uint16_t, int32_t, uint32_t, int64_t, uint64_t — fixed-width integer types (C99)
<stdbool.h>	bool, true, false — boolean support (C99)
<locale.h>	setlocale(), localeconv() — LC_ALL, LC_NUMERIC, LC_TIME
<setjmp.h>	setjmp(), longjmp(), jmp_buf — non-local jumps
<conio.h>	getch(), getche(), clrscr(), gotoxy(), kbhit() (NON-STANDARD — Borland/Turbo C only)
<inttypes.h>	PRId8, PRId16, PRId32, PRId64, PRIu32 — format macros for stdint types (C99)
<wchar.h>	wchar_t, wint_t, fwprintf, fwscanf, wcslen, wcscpy, wcscat (wide chars)
<complex.h>	complex type, creal(), cimag(), cabs(), carg() — complex arithmetic (C99)
<tgmath.h>	Type-generic math macros (select sqrt/sqrtf/sqrtl by type) (C99)
<stdatomic.h>	atomic_int, atomic_bool, atomic_load, atomic_store (C11)

Appendix B: Complete Answer Key

Correct answers are listed below. Detailed explanations for every question are in Appendix C.

Q# Ans	Q# Ans	Q# Ans	Q# Ans	Q# Ans
Q001 B	Q026 B	Q051 B	Q076 B	Q101 C
Q002 C	Q027 C	Q052 C	Q077 C	Q102 B
Q003 C	Q028 C	Q053 C	Q078 C	Q103 B
Q004 D	Q029 B	Q054 C	Q079 B	Q104 A
Q005 C	Q030 B	Q055 C	Q080 C	Q105 A
Q006 B	Q031 C	Q056 B	Q081 C	Q106 C
Q007 D	Q032 C	Q057 C	Q082 C	Q107 C
Q008 C	Q033 C	Q058 C	Q083 B	Q108 C
Q009 B	Q034 B	Q059 B	Q084 B	Q109 B
Q010 C	Q035 B	Q060 C	Q085 B	Q110 C
Q011 C	Q036 C	Q061 C	Q086 B	Q111 C
Q012 C	Q037 B	Q062 B	Q087 C	Q112 D
Q013 B	Q038 C	Q063 D	Q088 B	Q113 B
Q014 C	Q039 B	Q064 C	Q089 B	Q114 A
Q015 D	Q040 B	Q065 C	Q090 C	Q115 B
Q016 C	Q041 B	Q066 A	Q091 C	Q116 C
Q017 A	Q042 C	Q067 B	Q092 B	Q117 B
Q018 B	Q043 B	Q068 B	Q093 C	Q118 B
Q019 C	Q044 B	Q069 B	Q094 B	Q119 B
Q020 B	Q045 C	Q070 B	Q095 B	Q120 B
Q021 C	Q046 B	Q071 C	Q096 B	Q121 A
Q022 C	Q047 D	Q072 C	Q097 D	Q122 A
Q023 B	Q048 B	Q073 B	Q098 C	Q123 B
Q024 C	Q049 C	Q074 C	Q099 C	Q124 B
Q025 C	Q050 B	Q075 B	Q100 C	Q125 C

Appendix C: Detailed Explanations

Q1. 1. Which header file is required to use the printf() and scanf() functions in C?

Answer: B) stdio.h

stdio.h stands for Standard Input/Output. It declares printf(), scanf(), fprintf(), fscanf(), fopen(), fclose() and all I/O-related functions. It is the most commonly included header in C programs.

Q2. 2. What does the acronym stdio.h stand for?

Answer: C) Standard Input/Output

stdio.h literally expands to 'Standard Input/Output header file'. It provides the standard set of I/O facilities for C programs.

Q3. 3. Which header file provides the sqrt() and pow() functions?

Answer: C) math.h

math.h declares mathematical functions: sqrt(), pow(), floor(), ceil(), sin(), cos(), tan(), fabs(), log(), exp() etc. You must also link the math library with -lm on Linux.

Q4. 4. The function strlen() is declared in which header file?

Answer: D) string.h

string.h provides string-handling functions: strlen(), strcpy(), strcat(), strcmp(), strchr(), strstr(), memset(), memcpy() etc.

Q5. 5. Which header file must be included to use malloc() and free()?

Answer: C) stdlib.h

stdlib.h (Standard Library) provides memory management (malloc, calloc, realloc, free), process control (exit, abort), integer arithmetic (abs, div), and string conversion (atoi, atof, strtol).

Q6. 6. The isalpha() and isdigit() functions are declared in:

Answer: B) ctype.h

ctype.h (Character Type) declares character classification functions: isalpha(), isdigit(), isalnum(), isspace(), isupper(), islower(), toupper(), tolower() etc.

Q7. 7. Which header file is used for time-related functions like time() and clock()?

Answer: D) time.h

time.h declares the time_t type, struct tm, and functions: time(), clock(), difftime(), mktime(), strftime(), localtime(), gmtime() for date/time manipulation.

Q8. 8. To use assert() macro in C, which header file is needed?

Answer: C) assert.h

assert.h provides the assert() macro used for debugging. When NDEBUG is defined, assert() is disabled, making it safe to leave assertions in production code.

Q9. 9. The errno variable and error codes are declared in:

Answer: B) errno.h

errno.h declares the errno variable and error code macros like EDOM, ERANGE, ENOMEM etc. Functions set errno to indicate error conditions.

Q10. 10. Which header file provides signal handling in C?

Answer: C) signal.h

signal.h declares signal(), raise(), and signal constants like SIGINT, SIGTERM, SIGFPE, SIGSEGV for handling system signals.

Q11. 11. The va_list, va_start, and va_end macros used for variable argument lists are in:

Answer: C) stdarg.h

stdarg.h provides the mechanism for functions with variable number of arguments (variadic functions). It defines va_list type and va_start(), va_arg(), va_end() macros.

Q12. 12. Which header file contains the NULL macro and size_t type?

Answer: C) stddef.h

stddef.h defines common macros and types: NULL (null pointer constant), size_t (unsigned type for sizes), ptrdiff_t (pointer difference type), and offsetof() macro.

Q13. 13. The function exit() for program termination is declared in:

Answer: B) stdlib.h

stdlib.h declares exit(), atexit(), and abort(). EXIT_SUCCESS (0) and EXIT_FAILURE (1) constants are also defined there.

Q14. 14. Which header file provides setjmp() and longjmp() for non-local jumps?

Answer: C) setjmp.h

setjmp.h provides setjmp() and longjmp() for implementing non-local jumps (exception-like handling). setjmp() saves the calling environment, and longjmp() restores it.

Q15. 15. The FILE data type and file operations are defined in which header?

Answer: D) stdio.h

The FILE type, along with fopen(), fclose(), fread(), fwrite(), fseek(), ftell(), rewind() and macros like EOF, SEEK_SET, SEEK_CUR, SEEK_END are all in stdio.h.

Q16. 16. Which header provides integer limits like INT_MAX, INT_MIN, CHAR_MAX?

Answer: C) limits.h

limits.h defines implementation-specific limits for integer types: CHAR_BIT, CHAR_MIN, CHAR_MAX, SHRT_MAX, INT_MAX, INT_MIN, LONG_MAX, ULONG_MAX etc.

Q17. 17. The float limits FLT_MAX, DBL_MAX, and FLT_EPSILON are in:

Answer: A) float.h

float.h defines the characteristics of floating-point types: FLT_MAX, FLT_MIN, FLT_EPSILON (for float), DBL_MAX, DBL_MIN, DBL_EPSILON (for double), LDBL_MAX etc.

Q18. 18. Which header file is used for locale-specific behavior like setlocale()?

Answer: B) locale.h

locale.h provides setlocale() and localeconv() for locale-specific formatting of numbers, dates, and currencies. LC_ALL, LC_NUMERIC, LC_TIME are defined here.

Q19. 19. To use the wchar_t wide-character type, which header is needed?

Answer: C) wchar.h

wchar.h (added in C95/C99) provides wide-character types and functions: wchar_t, wint_t, fwprintf(), fwscanf(), wcslen(), wcscpy(), wcscat() etc.

Q20. 20. The stdbool.h header provides which feature?

Answer: B) Boolean type bool and constants true/false

stdbool.h (C99) defines the _Bool type (aliased as bool) and the constants true (1) and false (0), enabling C programs to use boolean variables naturally.

Q21. 21. Which header file contains the int8_t, int16_t, int32_t, int64_t fixed-width integer types?

Answer: C) stdint.h

stdint.h (C99) declares fixed-width integer types: int8_t, uint8_t, int16_t, uint16_t, int32_t, uint32_t, int64_t, uint64_t — ensuring consistent sizes across platforms.

Q22. 22. The complex.h header (C99) provides support for:

Answer: C) Complex number arithmetic

complex.h provides support for complex number arithmetic: complex type, creal(), cimag(), cabs(), carg(), cadd(), csub(), cmul(), cdiv() etc. It requires C99 or later.

Q23. 23. Which header file supports printing format macros like PRId32, PRIu64?

Answer: B) inttypes.h

inttypes.h (C99) extends stdint.h by adding format macros for printf/scanf: PRId8, PRId16, PRId32, PRId64, PRIu32 etc., ensuring portable formatted I/O of fixed-width integers.

Q24. 24. The conio.h header file provides functions like getch() and clrscr(). This header is:

Answer: C) A non-standard, compiler-specific header

conio.h (Console I/O) is NOT part of the ANSI/ISO C standard. It is Borland/Turbo C specific and not available in GCC/MSVC by default. getch(), clrscr(), gotoxy() are its common functions.

Q25. 25. Which of the following is the correct way to include a standard header file?

Answer: C) #include

Standard header files are included using angle brackets < >: #include . This tells the preprocessor to search in system/standard include directories. User-defined headers use double quotes.

Q26. 26. How is a user-defined header file included in a C program?

Answer: B) #include "myfile.h"

User-defined header files are included using double quotes: #include "myfile.h". The preprocessor first searches the current directory, then the system include directories.

Q27. 27. What is the recommended file extension for user-defined header files in C?

Answer: C) .h

The conventional extension for C header files is .h (e.g., myutils.h). This convention is universally recognized by C compilers and tools, though any extension technically works.

Q28. 28. Which preprocessor directive prevents a header file from being included more than once?

Answer: C) #pragma once or #ifndef guard

Include guards (#ifndef HEADER_H / #define HEADER_H / ... / #endif) or #pragma once prevent multiple inclusion. Without them, redefinition errors can occur when a header is included from multiple files.

Q29. 29. What is an include guard in a header file?

Answer: B) A set of preprocessor directives preventing multiple inclusion

An include guard uses #ifndef SYMBOL / #define SYMBOL / ... / #endif. The first time the header is included, SYMBOL is undefined so the body is processed and SYMBOL is defined. Subsequent inclusions find SYMBOL defined and skip the body.

Q30. 30. Which of the following is a correct include guard structure?

Answer: B) #ifndef MYHEADER_H\n#define MYHEADER_H\n...\n#endif

#ifndef checks if the macro is NOT defined. If not defined, it defines the macro and includes the header content. #endif closes the conditional block. This is the standard include guard pattern.

Q31. 31. What can typically be placed inside a user-defined header file?

Answer: C) Function prototypes, macros, structure definitions, and type definitions

Header files typically contain: function prototypes (declarations), macro definitions (#define), structure/union/enum definitions, typedef declarations, extern variable declarations, and inline function definitions.

Q32. 32. What is the main purpose of a header file in C?

Answer: C) To declare interfaces shared across multiple source files

Header files serve as the 'interface' to a module. They declare what is available (functions, types, macros) without revealing the implementation details, enabling modular programming and code reuse.

Q33. 33. If a user-defined header 'mymath.h' is in the same directory as the source file, which include is correct?

Answer: C) #include "mymath.h"

#include "mymath.h" searches first in the current directory (where the source file resides), making it the correct syntax when the header is in the same directory as the .c file.

Q34. 34. Placing function definitions (not prototypes) in a header file that is included by multiple .c files can cause:

Answer: B) Multiple definition errors at link time

If a function body is in a header and that header is included by multiple .c files, each .c file gets its own copy of the function. The linker sees multiple definitions of the same function and raises an error (unless declared inline or static).

Q35. 35. The #pragma once directive serves the same purpose as:

Answer: B) Include guards (#ifndef/#define/#endif)

#pragma once is a non-standard but widely supported alternative to include guards. It instructs the compiler to include the file only once per compilation unit, preventing multiple inclusions.

Q36. 36. In which file should the actual implementation (body) of functions be written?

Answer: C) In the .c source file

Function implementations (bodies) belong in .c source files. Header files (.h) contain only declarations (prototypes). This separation enables separate compilation and reuse of object files.

Q37. 37. Can a user-defined header file include another header file?

Answer: B) Yes, a header can include other headers

Yes. Headers can include other headers. For example, your myutils.h might #include so that any file including myutils.h automatically gets stdio.h. Include guards prevent any loops or duplication.

Q38. 38. Which keyword is used to declare a variable in a header that is defined in a .c file?

Answer: C) extern

The extern keyword tells the compiler 'this variable exists, but is defined elsewhere.' By putting extern int count; in the header, multiple files can reference the same global variable defined once in a .c file.

Q39. 39. What happens when the preprocessor encounters #include "config.h"?

Answer: B) It replaces the directive with the entire contents of config.h

The preprocessor literally replaces the #include directive with the complete text content of the specified file before compilation begins. This is purely textual substitution.

Q40. 40. A header file used across a large project should be placed in:

Answer: B) A dedicated include/ or headers/ directory accessible project-wide

In large projects, headers are placed in a dedicated include/ directory. The compiler's include path (-I flag in GCC) is set to point to this directory, allowing all source files to access shared headers.

Q41. 41. Which of the following correctly defines a constant PI in a user-defined header?

Answer: B) #define PI 3.14159

#define PI 3.14159 creates a preprocessor macro. When the code is compiled, all occurrences of PI in the source are textually replaced by 3.14159 before the compiler sees the code.

Q42. 42. Which of the following is NOT appropriate to place in a header file?

Answer: C) Non-inline function definitions with large bodies

Non-inline function definitions in headers cause multiple definition errors (one per translation unit that includes the header). Only inline functions, macros, type definitions, and extern declarations should go in headers.

Q43. 43. If two header files each define a struct with the same name and both are included, what occurs?

Answer: B) A redefinition error at compile time

C does not allow redefinition of a struct with the same tag in the same translation unit. Including two headers that each define struct Point { }; causes a compile-time 'redefinition' error. Include guards or namespacing the struct names prevent this.

Q44. 44. The convention of naming include guards using the header filename in uppercase with underscores (e.g., MYMATH_H) is:

Answer: B) A widely accepted best practice to avoid naming conflicts

Using FILENAME_H_ or _FILENAME_H as include guard macros is a widely followed convention. The C standard reserves identifiers starting with underscore for the implementation, so MYMATH_H (without leading underscore) is the common safe choice.

Q45. 45. Which of the following is the correct syntax to define a user-defined function prototype in a header?

Answer: C) int add(int a, int b);

A function prototype in C is simply the function signature followed by a semicolon: int add(int a, int b); Parameter names are optional in prototypes. The extern keyword is implied for function declarations.

Q46. 46. Including a header file with angle brackets vs double quotes differs in:

Answer: B) The search path order the preprocessor uses

Angle brackets <> tell the preprocessor to search standard/system include directories only. Double quotes "" tell it to first search the current directory (and user-specified -I paths), then fall back to system directories.

Q47. 47. Which of the following is a valid user-defined header filename?

Answer: D) Both B and C

Both my-utils.h and my_utils.h are valid filenames. Spaces in filenames cause issues with include directives and are strongly discouraged. Underscores and hyphens are both acceptable, though underscores (my_utils.h) are more conventional in C.

Q48. 48. What is a forward declaration in the context of header files?

Answer: B) Declaring an identifier before it is fully defined

A forward declaration announces the existence of a function or type without providing its full definition. For example, struct Node; declares that struct Node exists, allowing pointers to it before the full struct definition.

Q49. 49. In a project, config.h is included in 5 different .c files. Without include guards, this causes:

Answer: C) Possible multiple redefinition errors

Without include guards, each #include "config.h" in each .c file inserts the full content. If any two source files both include a header that defines a struct or extern, the linker/compiler may see duplicate definitions and report errors.

Q50. 50. Which of these correctly creates a user-defined header file for a simple calculator module?

Answer: B) calc.h with #include guards, prototypes (add, sub, mul, div), and any needed #defines

A proper calculator header (calc.h) should contain: include guards, function prototypes (int add(int,int); etc.), any relevant macros (#define MAX_INPUT 1000), and possibly struct definitions — but NOT function bodies or local variable definitions.

Q51. 51. What is the output of strlen("HELLO")?

Answer: B) 5

strlen() counts characters excluding the null terminator '\0'. "HELLO" has 5 characters (H,E,L,L,O), so strlen() returns 5. The string is stored as 6 bytes including '\0', but strlen reports 5.

Q52. 52. Which function copies a string, including the null terminator?

Answer: C) strcpy()

strcpy(dest, src) copies the null-terminated string src into dest, including the terminating '\0'. It returns dest. Care must be taken that dest has sufficient space; otherwise buffer overflow occurs.

Q53. 53. The function strcat(dest, src) does what?

Answer: C) Appends src to the end of dest

strcat() appends the string src (including its '\0') to the end of dest (overwriting dest's '\0'). It returns dest. The destination must have enough space for both strings plus the null terminator.

Q54. 54. What does strcmp("abc", "abc") return?

Answer: C) 0

strcmp() returns 0 when both strings are identical, a negative value when the first string is lexicographically less than the second, and a positive value when it is greater.

Q55. 55. Which function in stdio.h reads a formatted string from stdin?

Answer: C) scanf()

scanf() reads formatted input from stdin according to a format string. %d reads integers, %f reads floats, %s reads strings, %c reads characters. It returns the number of items successfully read.

Q56. 56. What does fopen("file.txt", "r") return if the file does not exist?

Answer: B) NULL

fopen() returns NULL when the file cannot be opened (file not found, permission denied etc.). Always check if fopen() returns NULL before using the file pointer to avoid undefined behavior.

Q57. 57. The fclose() function is used to:

Answer: C) Close an open file and flush its buffer

fclose() flushes any buffered output, closes the file, and releases the FILE structure. It returns 0 on success and EOF on error. Always call fclose() after file operations to prevent resource leaks.

Q58. 58. Which function checks if the end of file has been reached?

Answer: C) feof()

feof(fp) returns non-zero (true) when the end-of-file indicator for stream fp has been set, i.e., a previous read operation reached the end of the file.

Q59. 59. The function atoi() from stdlib.h converts:

Answer: B) ASCII string to integer

atoi() (ASCII to Integer) converts a string representation of an integer to an int. For example, atoi("42") returns 42. For better error handling, strtol() is preferred in modern code.

Q60. 60. Which math.h function returns the absolute value of a floating-point number?

Answer: C) fabs()

fabs() returns the absolute value of a double. For integers, abs() from stdlib.h is used. fabsf() works with float and fabsl() with long double. Using abs() on a float may give incorrect results.

Q61. 61. The ceil() function in math.h returns:

Answer: C) The smallest integer not less than x

ceil(x) returns the smallest integer value that is greater than or equal to x (rounds up). For example, ceil(2.3) = 3.0, ceil(-2.3) = -2.0. It returns a double.

Q62. 62. What does floor(3.7) return?

Answer: B) 3.0

floor(x) returns the largest integer value not greater than x (rounds down toward negative infinity). floor(3.7) = 3.0, floor(-3.7) = -4.0. Return type is double.

Q63. 63. Which function from stdlib.h allocates memory for an array of elements, initializing them to zero?

Answer: D) calloc()

calloc(n, size) allocates memory for n elements each of given size bytes and initializes ALL bytes to zero. malloc() allocates but does NOT initialize. calloc(10, sizeof(int)) allocates space for 10 integers, all set to 0.

Q64. 64. The realloc(ptr, new_size) function:

Answer: C) Resizes previously allocated memory, possibly moving it

realloc() attempts to resize the memory block at ptr to new_size bytes. It may return the same pointer (if resized in-place) or a new pointer (if moved). The old pointer becomes invalid if a new address is returned.

Q65. 65. What does free(ptr) do when ptr is NULL?

Answer: C) Does nothing (safe to call)

The C standard guarantees that free(NULL) is a no-op (does nothing). This makes it safe to call free on a pointer that may or may not have been allocated, without an explicit NULL check.

Q66. 66. Which function converts an integer to a string representation in C?

Answer: A) itoa() or sprintf()

itoa() is a non-standard function (available in some compilers). The standard approach is sprintf(buffer, "%d", number) which writes the integer as a string into buffer. snprintf() is safer as it limits the output size.

Q67. 67. The toupper() function from ctype.h:

Answer: B) Converts a single character to uppercase

toupper(c) converts a single lowercase character c to its uppercase equivalent. If c is not a lowercase letter, it is returned unchanged. To convert a string, loop through each character and apply toupper().

Q68. 68. Which function from stdio.h writes a character to a file?

Answer: B) putc() or fputc()

fputc(c, fp) writes the character c to the file stream fp. putc(c, fp) is typically a macro equivalent of fputc(). Both return the character written on success or EOF on error.

Q69. 69. The fgets(str, n, fp) function reads:

Answer: B) Up to n-1 characters from fp, adding '\n' and '\0'

fgets() reads at most n-1 characters from stream fp into str, stopping at newline (which IS stored in str) or EOF, then appends '\0'. It is safer than gets() which has no buffer size limit.

Q70. 70. What does putchar(c) do?

Answer: B) Writes character c to stdout

putchar(c) is equivalent to fputc(c, stdout). It writes a single character to standard output. It is declared in stdio.h and returns the character written on success or EOF on failure.

Q71. 71. The getchar() function:

Answer: C) Gets a single character from stdin

getchar() reads the next character from standard input (stdin). It is equivalent to fgetc(stdin). It returns the character as an unsigned char converted to int, or EOF at end-of-file.

Q72. 72. Which function uses a format string to write to a character array (string)?

Answer: C) sprintf()

sprintf(buffer, format, ...) works like printf() but writes the formatted output into the character array buffer instead of stdout. snprintf(buffer, size, format, ...) is safer as it limits output to size characters.

Q73. 73. The rand() function from stdlib.h returns:

Answer: B) A pseudo-random integer in [0, RAND_MAX]

rand() returns a pseudo-random integer between 0 and RAND_MAX (at least 32767). The sequence is deterministic; seed it with srand(time(NULL)) for different sequences each run.

Q74. 74. Which function from time.h returns the current calendar time?

Answer: C) time()

time(NULL) returns the current time as a time_t value (typically seconds since the Unix epoch, Jan 1 1970). clock() returns CPU time used, not wall-clock time.

Q75. 75. The memset(ptr, value, n) function sets:

Answer: B) n bytes starting at ptr to value

memset() sets the first n bytes of the memory area pointed to by ptr to the byte value (as unsigned char). It is commonly used to initialize arrays/buffers: memset(arr, 0, sizeof(arr)) zeros the array.

Q76. 76. Which function searches for the first occurrence of a character in a string?

Answer: B) strchr()

strchr(str, c) returns a pointer to the first occurrence of character c in string str, or NULL if not found. strrchr() finds the LAST occurrence. strstr() searches for a substring, not a single character.

Q77. 77. The pow(x, y) function computes:

Answer: C) x raised to the power y

pow(x, y) returns x^y (x raised to the power y) as a double. pow(2.0, 10.0) = 1024.0. Both arguments must be double. For integer powers, explicit multiplication in a loop avoids floating-point inaccuracies.

Q78. 78. Which header provides qsort() for sorting arrays?

Answer: C) stdlib.h

qsort() is declared in stdlib.h. It sorts an array using a comparison function provided by the programmer: qsort(arr, n, sizeof(int), compare). The comparison function returns negative, zero, or positive values.

Q79. 79. The bsearch() function in stdlib.h:

Answer: B) Does binary search on a sorted array

bsearch() performs binary search on a sorted array. Like qsort(), it takes a comparison function. It returns a pointer to the matching element or NULL if not found. The array MUST be sorted first.

Q80. 80. What does isspace(' ') return?

Answer: C) Non-zero (true)

isspace(c) from ctype.h returns non-zero (true) if c is a whitespace character: space ' ', tab '\t', newline '\n', carriage return '\r', form feed '\f', or vertical tab '\v'. isspace(' ') returns non-zero.

Q81. 81. Which stdio.h function repositions the file pointer to the beginning of a file?

Answer: C) rewind(fp)

rewind(fp) moves the file position indicator to the beginning (offset 0) of the file stream fp and clears the end-of-file and error indicators. It is equivalent to fseek(fp, 0L, SEEK_SET).

Q82. 82. The fseek(fp, offset, whence) function with SEEK_CUR as whence moves the file pointer:

Answer: C) Relative to the current position

SEEK_CUR moves the file pointer offset bytes relative to the current position. SEEK_SET moves to an absolute position from the file beginning. SEEK_END moves relative to the end of file.

Q83. 83. What does ftell(fp) return?

Answer: B) Current position of the file pointer as a long int

ftell() returns the current value of the position indicator for stream fp as a long int (byte offset from the beginning). It returns -1L on error. Combined with fseek(), it is used to determine file size.

Q84. 84. The perror(str) function from stdio.h:

Answer: B) Prints a system error message corresponding to errno

perror(str) prints str followed by a colon and the error message corresponding to the current value of errno (set by library functions on error). For example, after a failed fopen(), perror("Error") might print 'Error: No such file or directory'.

Q85. 85. Which function from string.h compares two strings case-insensitively?

Answer: B) stricmp() / strcasecmp()

strcasecmp() (POSIX/Linux) or stricmp() (Windows/Borland) compare strings ignoring case. The standard strcmp() is case-sensitive. strcoll() compares using locale rules. strncmp() compares at most n characters.

Q86. 86. The strtok(str, delimiters) function is used for:

Answer: B) Splitting a string into tokens using delimiter characters

strtok() tokenizes a string by replacing delimiter characters with '\0' and returning pointers to each token. First call: strtok(str, delim). Subsequent calls: strtok(NULL, delim). It modifies the original string.

Q87. 87. The return type of malloc() is:

Answer: C) void*

malloc() returns void* (a generic pointer). In C, void* is automatically converted to any pointer type upon assignment, so explicit casting is not required (unlike C++). If allocation fails, it returns NULL.

Q88. 88. Which function from math.h computes the natural logarithm?

Answer: B) log()

log(x) computes the natural logarithm (base e) of x. log10(x) computes the base-10 logarithm. log2(x) (C99) computes the base-2 logarithm. All return double; x must be positive.

Q89. 89. What is the purpose of fflush(stdout)?

Answer: B) Forces all buffered output for stdout to be written immediately

fflush() forces any buffered data for the given stream to be written to its underlying file or device. fflush(stdout) ensures pending printf output appears immediately, which is useful before reading input or debugging.

Q90. 90. The abs() function in stdlib.h works on which data type?

Answer: C) int

abs() in stdlib.h operates on int values. For long integers, use labs(). For long long, use llabs() (C99). For floating-point, use fabs() (for double), fabsf() (for float), fabsl() (for long double) from math.h.

Q91. 91. Which header provides the system() function to execute OS commands?

Answer: C) stdlib.h

system(command) from stdlib.h passes command to the host OS shell for execution (e.g., system("ls") on Linux, system("dir") on Windows). It returns the command's exit status.

Q92. 92. The localtime() function from time.h returns a pointer to:

Answer: B) A struct tm containing broken-down local time

localtime(time_t*) converts a time_t value to a struct tm representing local time with fields: tm_year (years since 1900), tm_mon (0-11), tm_mday (1-31), tm_hour, tm_min, tm_sec etc.

Q93. 93. Which function from string.h finds the first occurrence of a substring?

Answer: C) strstr()

strstr(haystack, needle) returns a pointer to the first occurrence of string needle within string haystack, or NULL if not found. It searches for the entire substring, unlike strchr() which searches for a single character.

Q94. 94. What is RAND_MAX?

Answer: B) The maximum value rand() can return

RAND_MAX is a macro defined in stdlib.h that represents the maximum value that rand() can return. The C standard guarantees it is at least 32767. The actual value is implementation-specific (often 32767 or 2147483647).

Q95. 95. Which function safely limits the number of characters copied to prevent buffer overflow?

Answer: B) strncpy()

strncpy(dest, src, n) copies at most n characters from src to dest. Unlike strcpy(), it does not overflow the buffer. However, if src has n or more characters, strncpy() may NOT null-terminate dest, requiring explicit handling.

Q96. 96. The difftime(t2, t1) function returns:

Answer: B) The difference t2-t1 in seconds as a double

difftime(time2, time1) computes the difference between two time_t values and returns it as a double representing the number of seconds elapsed from time1 to time2.

Q97. 97. Which ctype.h function checks if a character is a decimal digit?

Answer: D) isdigit()

isdigit(c) returns non-zero if c is a decimal digit character ('0' through '9'). isalpha() checks for letters, isalnum() checks for letters or digits, isxdigit() checks for hexadecimal digits.

Q98. 98. The function strrev() that reverses a string is:

Answer: C) Non-standard (available in some compilers like Borland)

strrev() is NOT part of the ANSI/ISO C standard. It is available in Borland C/Turbo C. To reverse a string in standard C, implement it manually using a loop or use other string functions.

Q99. 99. Which function writes formatted data to a file stream?

Answer: C) fprintf()

fprintf(fp, format, ...) works like printf() but writes to the specified file stream fp instead of stdout. fprintf(stderr, ...) is commonly used to write error messages to standard error.

Q100. 100. What does memcpy(dest, src, n) do?

Answer: C) Copies n bytes from src to dest (undefined behavior if overlapping)

memcpy() copies exactly n bytes from the memory area src to dest. If the source and destination overlap, behavior is undefined — use memmove() instead, which correctly handles overlapping regions.

Q101. 101. The #include directive is processed by the:

Answer: C) Preprocessor

The C preprocessor runs before the compiler. It handles all directives starting with #: #include (file inclusion), #define (macros), #ifdef/#ifndef (conditional compilation), #pragma (compiler-specific instructions).

Q102. 102. Which of the following is true about header files and compilation?

Answer: B) Header files are textually inserted into source files before compilation

The preprocessor replaces each #include directive with the exact text contents of the specified file. The compiler then compiles the expanded source (called a translation unit) as a single unit. Header files do NOT produce object files.

Q103. 103. What is a translation unit in C?

Answer: B) A single .c file after preprocessor expansion (including all headers)

A translation unit is the source code seen by the compiler: a single .c source file plus all the headers it includes (after preprocessing). Each translation unit is compiled independently into an object file (.o).

Q104. 104. The One Definition Rule (ODR) in C states:

Answer: A) Every variable must be defined exactly once across all translation units

Each global variable and non-inline function must have exactly ONE definition across all translation units. Multiple declarations (with extern) are allowed, but multiple definitions cause linker errors. This is the One Definition Rule.

Q105. 105. Which of the following correctly represents the order of C compilation stages?

Answer: A) Preprocessing → Compilation → Assembly → Linking

The standard C compilation pipeline: (1) Preprocessing — handles #include, #define, #ifdef; (2) Compilation — translates C to assembly; (3) Assembly — converts assembly to machine code (object file); (4) Linking — combines object files and libraries into an executable.

Q106. 106. An inline function defined in a header file:

Answer: C) Is allowed in headers; each translation unit gets its own copy

The inline keyword is an exception to ODR. Inline function definitions may appear in multiple translation units (each sees the same definition via the header). The compiler may substitute the function body at the call site to avoid function call overhead.

Q107. 107. Which header file contains the macro offsetof(type, member)?

Answer: C) stddef.h

offsetof(type, member) is a macro in stddef.h that returns the byte offset of member within a struct or union type. It is extensively used in container_of and similar patterns in systems programming.

Q108. 108. When you use gcc -E file.c, what does gcc produce?

Answer: C) The preprocessed output (C code after #includes expansion)

gcc -E stops after the preprocessing stage and prints the preprocessed source to stdout. This is useful for debugging: you can see exactly what text the compiler receives after all #include and #define expansions.

Q109. 109. The C standard header iso646.h provides:

Answer: B) Alternative spellings of logical/bitwise operators

iso646.h (C94) defines alternative tokens for operators: and (&&), or (||), not (!), xor (^), bitand (&), bitor (|), and_eq (&=), or_eq (|=), xor_eq (^=), not_eq (!=). Useful where keyboard characters are limited.

Q110. 110. #pragma once is:

Answer: C) A non-standard but widely supported compiler extension

#pragma once is not part of the C standard (which defines no mandatory #pragmas), but is supported by GCC, Clang, MSVC and most modern compilers. It is simpler than include guards but not 100% portable to obscure compilers.

Q111. 111. The __FILE__ and __LINE__ predefined macros are declared in:

Answer: C) They are compiler-predefined, not from any header

__FILE__ (current filename as string), __LINE__ (current line number as int), __DATE__, __TIME__, and __func__ (C99, current function name) are predefined macros built into the C preprocessor — no header needed.

Q112. 112. Which header file should be included to use POSIX threads (pthreads)?

Answer: D) Both B (POSIX) and C (C11 standard)

POSIX systems use pthread.h (#include) with -lpthread linking. The C11 standard introduced its own threading in threads.h (). Both exist but serve slightly different contexts.

Q113. 113. In C, the header file extension .h is:

Answer: B) A convention; any extension works technically

The .h extension is a widely followed convention but NOT enforced by the language. The preprocessor doesn't care about the extension — it simply inserts the file contents. However, deviating from .h causes confusion and should be avoided.

Q114. 114. What is the purpose of the extern "C" declaration in a header file?

Answer: A) It allows the header to be used in both C and C++

In C++ code, functions in a C header need extern "C" to prevent C++ name mangling, ensuring proper linkage with C-compiled object files. This is typically wrapped in #ifdef __cplusplus so the header works in both C and C++.

Q115. 115. The tgmath.h header (C99) provides:

Answer: B) Type-generic math macros that select correct math functions based on argument type

tgmath.h provides type-generic macros (using _Generic in C11, or compiler extensions in C99) that automatically select the appropriate math function variant: sqrt() vs sqrtf() vs sqrtl() based on the argument type.

Q116. 116. Which of the following header files provides support for atomic operations (C11)?

Answer: C) stdatomic.h

stdatomic.h (C11) provides atomic types (atomic_int, atomic_bool etc.) and atomic operations (atomic_load, atomic_store, atomic_fetch_add etc.) for lock-free thread-safe programming.

Q117. 117. The function snprintf(buf, size, format, ...) compared to sprintf() is:

Answer: B) Safer — it limits output to size-1 characters, preventing buffer overflow

snprintf() is the safe version of sprintf(). It writes at most size-1 characters into buf and always null-terminates. sprintf() has no size limit and can cause buffer overflows — snprintf() is strongly preferred in modern C.

Q118. 118. When multiple header files are included, the order of inclusion:

Answer: B) Can matter if one header depends on types or macros defined in another

Include order can matter. If mylib.h uses types from stdint.h, then stdint.h must be included first (or mylib.h should include stdint.h itself). Well-designed headers are self-contained — they include all their own dependencies.

Q119. 119. The _Generic keyword introduced in C11 is used in headers to:

Answer: B) Create type-generic macros that select different expressions based on type

_Generic(expr, type1: val1, type2: val2, ...) is a C11 compile-time selection expression. It enables type-generic macros in headers, used in tgmath.h to select sqrt(), sqrtf(), or sqrtl() based on argument type.

Q120. 120. A header file should be self-contained, meaning:

Answer: B) It compiles correctly if included first, without requiring prior includes

A self-contained header includes all the headers it depends on. If mylib.h uses size_t, it should #include itself, not rely on the user having already included it. This prevents subtle errors and ordering problems.

Q121. 121. The MATCH: Which header goes with each function?
(i) printf — (ii) malloc — (iii) strlen — (iv) sqrt

Answer: A) stdio.h, stdlib.h, string.h, math.h

printf() → stdio.h (Standard I/O); malloc() → stdlib.h (Standard Library); strlen() → string.h (String operations); sqrt() → math.h (Mathematical functions). This mapping is fundamental knowledge.

Q122. 122. MATCH: Header file with its primary purpose. (i) ctype.h — (ii) errno.h — (iii) assert.h — (iv) limits.h

Answer: A) Character classification, Error codes, Debugging assertions, Integer limits

ctype.h → character classification (isalpha, isdigit); errno.h → error code variable and constants; assert.h → assert() debugging macro; limits.h → integer type range constants (INT_MAX etc.).

Q123. 123. CONCEPTUAL: Why should header files NOT contain global variable definitions (without extern)?

Answer: B) Including the header in multiple .c files creates multiple definitions, causing linker errors

If int count = 0; (a definition) appears in a header included by 3 .c files, each .c file defines its own count. The linker sees 3 definitions of the same global symbol and raises a 'multiple definition' error. Use extern int count; in the header and define it in exactly one .c file.

Q124. 124. CONCEPTUAL: What is the difference between declaration and definition in C?

Answer: B) Declaration introduces an identifier; definition also allocates storage/provides body

Declaration: announces a name and its type to the compiler (e.g., extern int x; or int add(int,int);) — no storage allocated. Definition: actually creates the entity (e.g., int x = 5; or a function with its body) — storage is allocated. Headers declare; source files define.

Q125. 125. CONCEPTUAL: Which statement about #include and the preprocessor is CORRECT?

Answer: C) The preprocessor replaces #include with file contents before compilation

The preprocessor (phase 1 of compilation) handles #include by reading the specified file and replacing the #include line with that file's entire textual content. This expanded source is then passed to the compiler. No separate compilation of headers occurs (unless using precompiled headers).

Topics Covered in This Prompt Series: